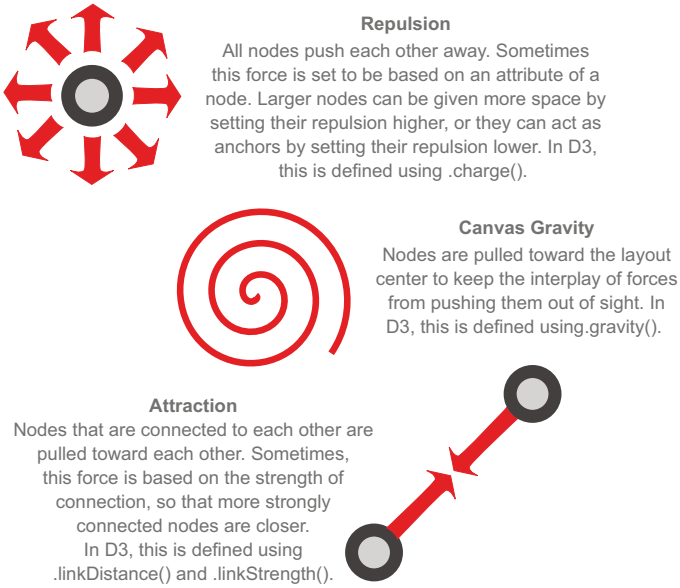


Figure 6.7 The forces in a force-directed algorithm: repulsion, gravity, and attraction. Other factors, such as hierarchical packing and community detection, can also be factored into force-directed algorithms, but these features are the most common. Forces are approximated for larger networks to improve performance.

6.2.1 Creating a force-directed network diagram

The `force()` layout you see initialized in listing 6.7 has some settings you've already seen before. The most obvious is `size()`, which uses an array containing the width and height of our layout region to calculate the necessary force settings. The `nodes()` and `links()` settings are the same as for the Sankey layout in chapter 5. They take, as you'd expect, arrays of data that correspond to the nodes and links. We're creating our own source and target references in our links array, just like we did with the arc diagram, and that's the formatting that `force()` expects. It also accepts integer values where the integer values correspond to the array position of a node in the nodes array, like the formatting of data for the Sankey diagram links array from chapter 5. As you can see in the following listing, the one setting that's new is `charge()`, which determines how much each node pushes away other nodes. There's also a new event listener, `"tick"`, that needs to get associated with a tick function that updates the position of your nodes and edges.

Listing 6.7 Force layout function

```
function forceDirected() {
   queue()
   .defer(d3.csv, "nodelist.csv")
   .defer(d3.csv, "edgelist.csv")
```